

Data Structures Lab

Array-Based Programs: Sorting · Stack · Queue

1. Insertion Sort

Algorithm:

1. Start from index 1.
2. Pick current element as 'key'.
3. Compare key with elements before it (moving left).
4. Shift larger elements one position to the right.
5. Insert key at the correct position.
6. Repeat for all elements till end of array.

C Code:

```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n, arr[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    insertionSort(arr, n);
    printf("Sorted: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

```
}
```

2. Bubble Sort

Algorithm:

1. Repeat n-1 passes over the array.
2. In each pass, compare adjacent elements.
3. If left element > right element, swap them.
4. After each pass, the largest unsorted element reaches its correct position.
5. Stop early if no swaps occur in a pass (already sorted).

C Code:

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n, arr[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    bubbleSort(arr, n);
    printf("Sorted: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

3. Merge Sort

Algorithm:

1. Divide the array into two halves from the middle.
2. Recursively sort the left half.
3. Recursively sort the right half.
4. Merge the two sorted halves into one sorted array.
5. During merge, compare elements one by one and place the smaller first.
6. Copy remaining elements of either half at the end.

C Code:

```
#include <stdio.h>

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int n, arr[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    mergeSort(arr, 0, n - 1);
    printf("Sorted: ");
    for (int i = 0; i < n; i++)
```

```
        printf("%d ", arr[i]);  
    return 0;  
}
```

4. Stack using Array

Algorithm:

1. Use an array and a variable 'top', initialized to -1.
2. Push: Check overflow (top == MAX-1), then increment top and insert arr[top] = val.
3. Pop: Check underflow (top == -1), then return arr[top] and decrement top.
4. Peek: Return arr[top] without modifying top.
5. isEmpty: Return true if top == -1.

C Code:

```
#include <stdio.h>  
  
#define MAX 100  
  
int stack[MAX], top = -1;  
  
void push(int val) {  
    if (top == MAX - 1) { printf("Overflow\n"); return; }  
    stack[++top] = val;  
}  
  
int pop() {  
    if (top == -1) { printf("Underflow\n"); return -1; }  
    return stack[top--];  
}  
  
int peek() {  
    if (top == -1) { printf("Empty\n"); return -1; }  
    return stack[top];  
}  
  
int main() {  
    int n, val, choice;  
    printf("Enter number of elements to push: ");  
    scanf("%d", &n);  
    for (int i = 0; i < n; i++) {  
        printf("Enter element %d: ", i + 1);  
        scanf("%d", &val);  
        push(val);  
    }  
}
```

```

    }
    printf("Top: %d\n", peek());
    printf("Pop: %d\n", pop());
    printf("Top after pop: %d\n", peek());
    return 0;
}

```

5. Queue using Array

Algorithm:

1. Use an array with two variables: front and rear, both initialized to -1.
2. Enqueue: Check overflow (rear == MAX-1). Set front=0 if first element. Increment rear, insert arr[rear] = val.
3. Dequeue: Check underflow (front == -1 or front > rear). Return arr[front] and increment front.
4. Reset front and rear to -1 when queue becomes empty (front > rear).
5. Front element: arr[front]. Rear element: arr[rear].

C Code:

```

#include <stdio.h>
#define MAX 100

int queue[MAX], front = -1, rear = -1;

void enqueue(int val) {
    if (rear == MAX - 1) { printf("Overflow\n"); return; }
    if (front == -1) front = 0;
    queue[++rear] = val;
}

int dequeue() {
    if (front == -1 || front > rear) { printf("Underflow\n"); return -1; }
    int val = queue[front++];
    if (front > rear) front = rear = -1;
    return val;
}

int main() {
    int n, val;
    printf("Enter number of elements to enqueue: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &val);
    }
}

```

```
        enqueue(val);  
    }  
    printf("Dequeue: %d\n", dequeue());  
    printf("Dequeue: %d\n", dequeue());  
    printf("Front element: %d\n", queue[front]);  
    return 0;  
}
```

All the best for your exam! ■